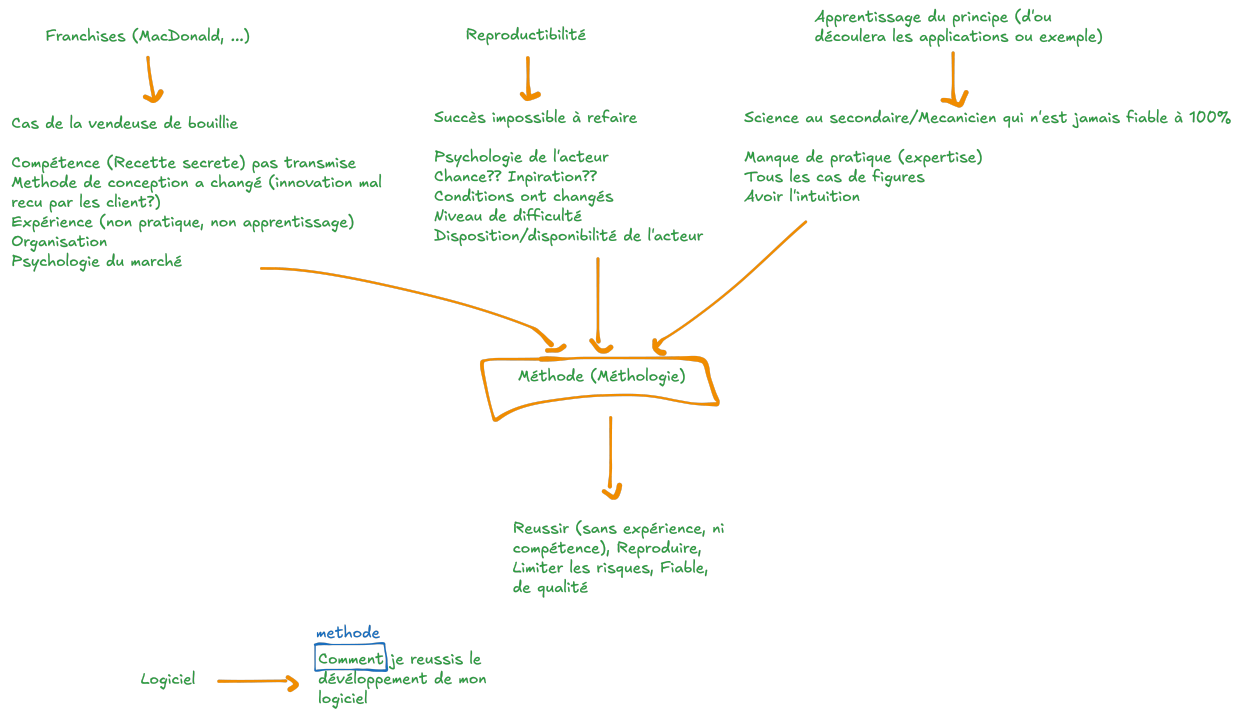


SDLC



Chap1. Introduction au SDLC

SDLC = Software Development Life Cycle $\Rightarrow$ cycle de vie de développement de logiciel

- Méthodologie $\Rightarrow$ comment développer efficacement un logiciel

Introduction

1. Logiciel ?

1.1. Brainstorming

un (ensemble) programme/procédure capable de :

= suite ordonnée d'instructions (ou algorithme)

- Faire fonctionner un système (ordinateur ou appareil électronique)
- Régler/accomplir un problème (ou des tâches spécifiques - traitement de données)
- automatiser des tâches



Logiciel = Une suite d'instruction pour dire à l'ordinateur ou à la machine que faire

De façon générale, on peut considérer que Logiciel = Programme=Application

Mais il existe souvent quelques spécificités

	Algorithme	Programme	Application	Logiciel	Progiciel
Definitions	Une suite d'instructions décrivant la résolution d'un problème	Une suite d'instructions exécutable sur ordinateur ou une machine	Programme ou Logiciel avec une interface graphique	Un ensemble de Programmes	Plusieurs logiciels ensemble pour la gestion (comptabilité, finance, ressource humaine,.. ERP

Différences	Absence d'ordinateur/ de machine	Fait une tâche spécifique		Effectue plusieurs tâches qui concourent à résoudre un/des problèmes	
				(Dev, Beta, Release/production)	
Exemple	Comment résoudre une équation du second degré	L'écrire dans un langage machine (Python)	Mettre une interface ou il y a un formulaire précisant les coefficients a, b et c et un bouton résoudre avec affichage du résultat	Résoudre plusieurs équations et inéquations dans R	Un Progiciel permettant la gestion d'un cursus universitaire orienté mathématique (logiciel d'algèbre, géométrie, équations, numérique, chimie, physiques)

1.2. Typologie

Trois grandes familles de logiciel

Système D'exploitation	Logiciel Machine (Drivers)	Logiciel d'utilité
Windows, Linux, MacOS, Android, FreeRTOS	Arduino, Logiciel déployé sur les appareils ou équipements électroniques	Tous le reste (Desktop, Web ou Mobile) - Word, Excel, Photoshop, Facebook, ...

1.3. Un bon logiciel ?

- qui respecte le cahier de charge
- critères qualité :
 - assez rapide
 - fiabilité (fiable = confiance) = sans bugs

Bon algo =

Correctness (il fait ce qu'on lui instruit de faire) +

Completness (prend en compte tous les cas) +

Efficient (**Rapidité**=temps d'exécution, **Stockage**=mémoire utilisée)



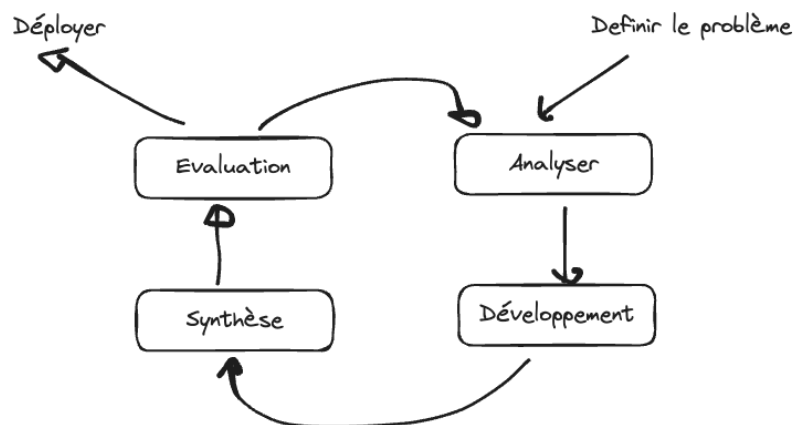
Bon logiciel = ensemble de bon algorithmes + tests (de validation et d'intégration)

Logiciel de qualité = Logiciel qui respecte des critères bien définis de qualité (fonctionnalité, performance, fiabilité, maintenance, utilisation, portabilité)

1.4. Les étapes de développement d'un logiciel

Comment résoudre un problème ?

- Décrire/Identifier le problème
- Analyser le problème (Décomposer le problème, trouver les causes, attaquer les causes, les solutions possibles)
- Résolution du problème (décider)



Importance d'avoir de suivre une méthodologie ?

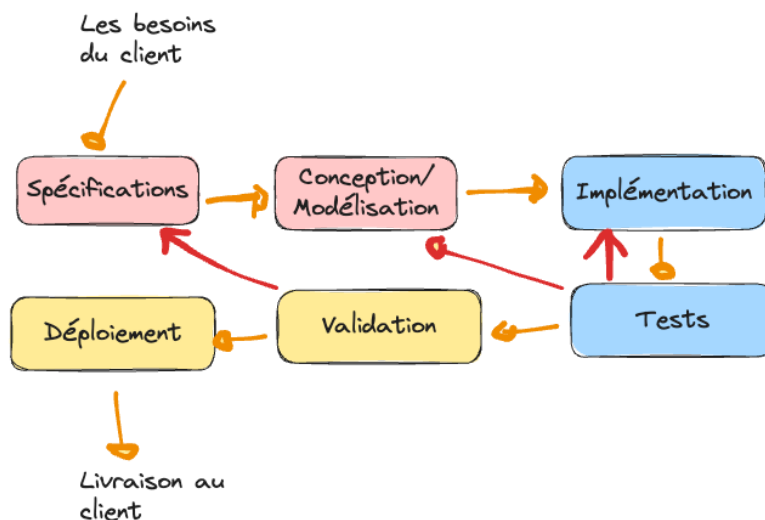
- Aboutir une bonne résolution de problème
 - Avoir un suivi assidu
 - Gagner du temps, l'argent
 - Plus performant
 - reproductibilité
-
- 3 exemples :

- Faire quelque chose une fois et ne plus pouvoir le faire, Ne pas pouvoir expliquer ce qui est fait, ⇒ inattention à la méthodologie, ⇒ **Reproductibilité impossible/difficile, maintenance difficile**
- L'explosion d'Ariane 5 en 1996 à cause d'une erreur logicielle ⇒ il y a trop de chose à faire et à vérifier qui peuvent être complexes les unes que les autres ⇒ **et une absence de méthodologie (étape par étape) peut conduire à des désastres.**
- Les franchises (MacDonald, Quick, Burger king ...) = développement de la méthodologie partout/ la recette magique ⇒ **un gain énorme en argent**

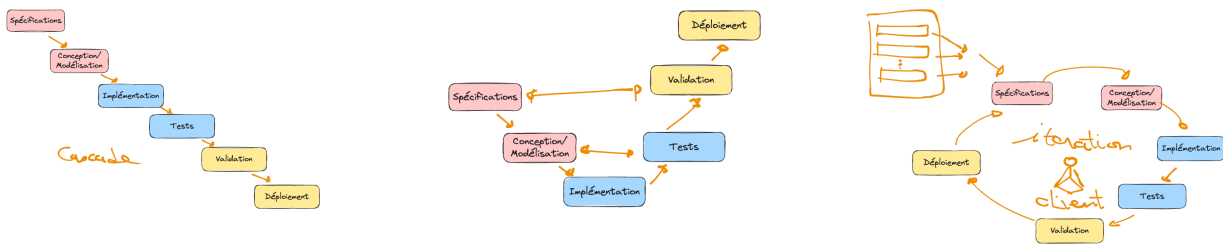


Développer un bon logiciel (et être sûre qu'il est performant, correct, complet et efficace) passe par le suivi d'un processus/méthodologie de façon rigoureuse

Les étapes standards de développement d'un logiciel

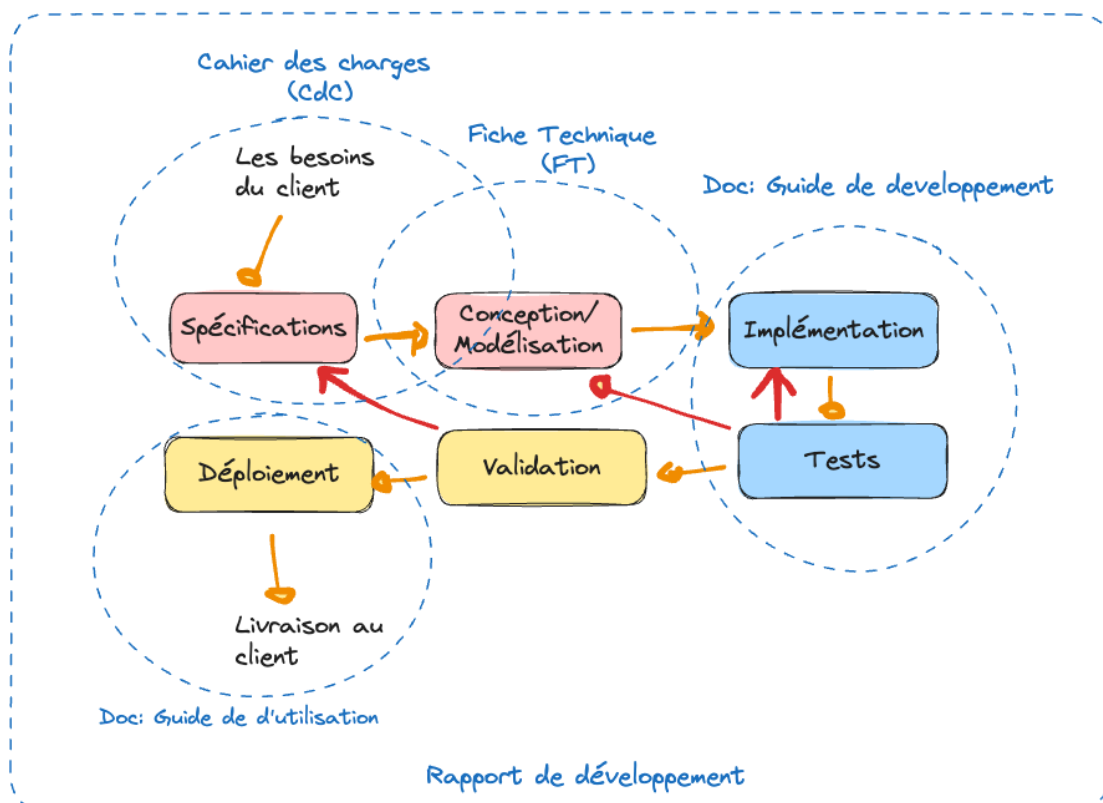


Toutes les méthodologies se baseront sur ce schéma standard avec des spécificités en fonction des problèmes à résoudre dans l'organisation du développement.



1.5. La documentation autour du développement d'un logiciel

- **le cahier des charges** (qui clarifie les besoins du client et constitue le contrat réel qui lie le développeur au client)
- **la fiche technique** (qui contient l'ensemble des éléments de modélisation permettant de traduire les spécifications en modèle de développement viable, souvent en utilisant un langage de modélisation comme UML, Merise)
- **Guide de développement** (qui reprend les choix techniques et comment chaque module est testé, comment le code est structuré, déploiement, ...)
- **Guide d'utilisation** (l'installation, l'utilisation - détailler les fonctionnalités aux utilisateurs étape par étape)

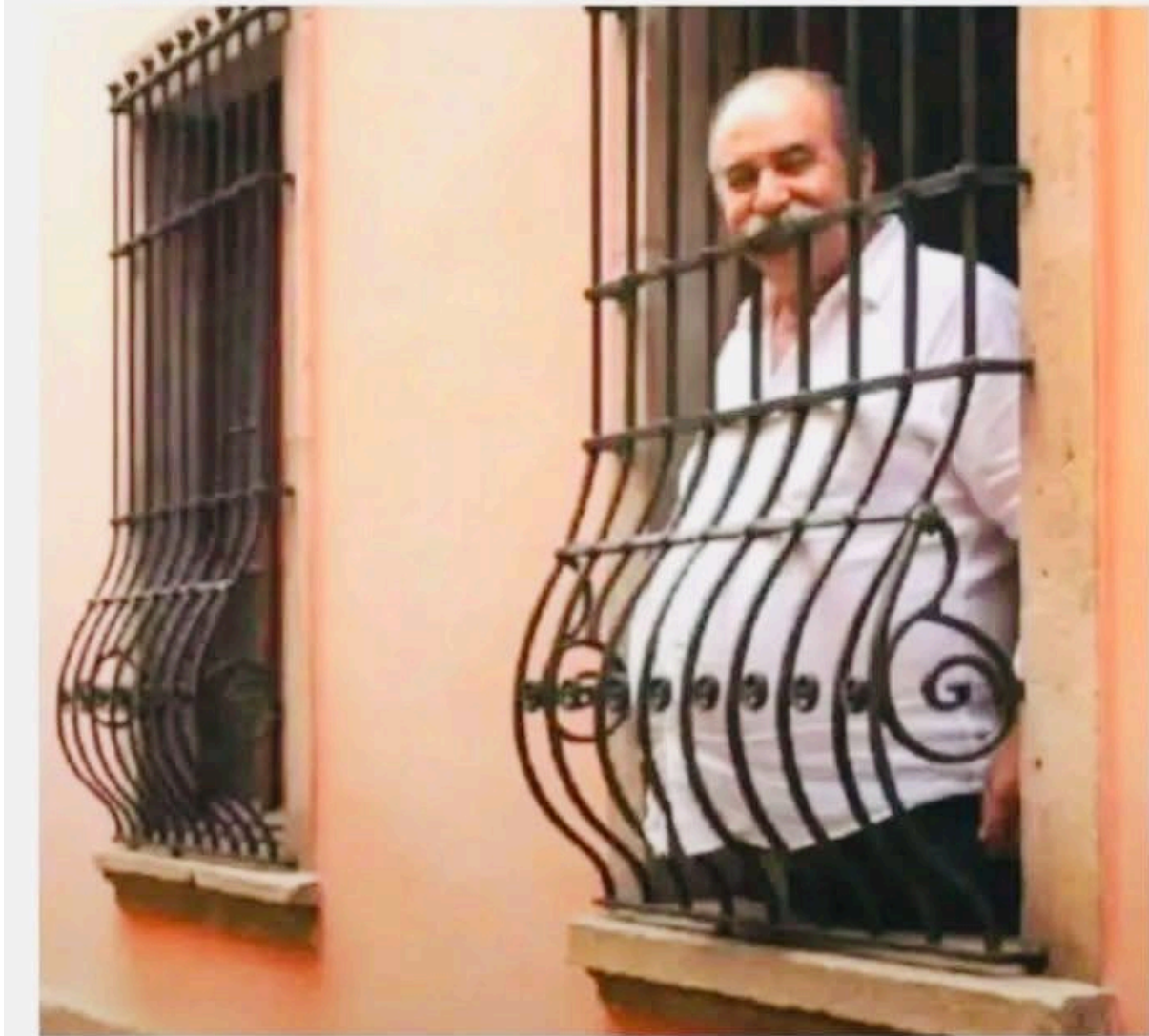




Le rapport de développement est souvent une synthèse de tous les documents du logiciel

Chap2. Cahier de charge et Fiche Technique

Understanding the client's requirements is a key to a successful project 🗝️



1. Cahier de charge

Est ce document qui permet de **clarifier** les besoins de client et clairement définir les spécifications du problèmes sur lesquels on s'entendra avec le client



Besoin du client > Analyse + pose questions + consulte les archives + réunions (re)cadrage > CdC

1.1. Les parties du CdC (le cœur)

1. **Objectif du projet** (clairement définir le projet, lui donner un titre + contexte, constats, une justification)
2. **Spécifications** (Spécifications fonctionnelles = les fonctionnalités du système + exemples, Spécifications non fonctionnelles = sécurité, convivialité, ergonomie, ...)

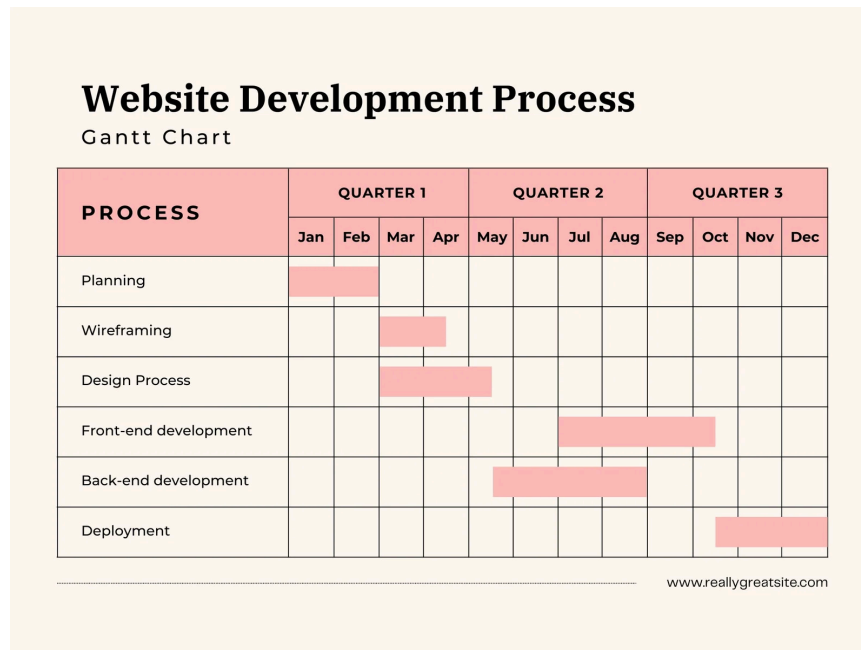


on peut mettre un petit diagramme de cas d'utilisation, **même si cela n'est pas forcément recommandé parce que trop technique pour le client**

3. Contraintes

- a. Client : doit payer, doit mettre à disposition une salle, doit mettre à disposition un serveur, mettre à disposition les employés pour enquêtes, ...
 - b. Prestataire : réalisation un logiciel qui répond aux besoins du client + (qualité), finir à temps, livrer le produit,
4. **Livrables** (spécifier clairement en tiret et avec descriptif si nécessaire les livrables, cette partie peut servir de contenu pour le bon de livraison, préciser s'il faut lui donner les droits ou pas, le code, les documents, les rapports, ...)
 5. **Chronogramme** (Dire clairement le délai de réalisation du travail avec une date de début et une date de fin, accompagné tout au moins d'un calendrier de

réalisation) = un diagramme de Gantt



6. Signature

7. *Maquettes (un ou 2 mock-up du logiciel, permettant aussi de discuter de la charte graphique)

8. * Le budget ou l'offre financière

a. Ne jamais juste donner le montant (répartir l'offre en plusieurs modules, incluant parfois même des alternatives,)

b. Produire s'il faut un document d'accompagnement en Excel qui montre le détail (attention : **le diable est dans le détail**)

2. Fiche Technique

Reprend généralement la modélisation du projet sous 2 aspects spécifiques :
l'architecture générale et la conception détaillée



Il est très important d'avoir en tête qu'une fiche technique **DOIT** permettre la réalisation complète du logiciel sans que le développeur ait vraiment besoin de contacter le client ou de consulter le cahier des charges.

- L'architecture générale de développement (MVC, Design pattern, Paradigme, API + interaction, RestFull, ..)
- Diagrammes UML par exemples : On aura au minimum
 - Cas d'utilisation (pour les fonctionnalités)
 - Classe pour la gestion des données
 - Objet (pour l'exemplification des classes et méthodes)
 - Séquences (et/ou d'état transition) - normalement au minimum pour chaque cas d'utilisation
- Les spécifications et choix logiciels
- Les maquettes - normalement au minimum pour chaque cas d'utilisation
- Gestion des données et contenu